Name: Khatri Yashvi Himanshubhai
Roll No.: 202351067
Course: CS362 (Distributed and Parallel Computing Lab)

Lab 8 Assignment

Aim: Simulates all five nodes, reads a seed.json file containing initial clock values, a tolerance threshold θ, and a scrambled queue of incoming messages for each node. You must use delays (e.g., pausing for fractions of a second) to stream the events to the terminal in real-time. The user should watch as nodes poll each other, messages traverse the network, and causal buffers hold and release data.
Step 1: Node 0 acts as the coordinator. It calculates a trusted time average and issues corrections to bring the clocks together.
Step 2: Each node processes its queue of incoming messages. If a message arrives too early (missing its causal history), the node must hold it in a buffer. As missing messages arrive, the buffer must be recursively checked and emptied.
Step 3: After processing all messages, each node calculates a final numerical payload to send to the portal.

Code:

```
import json
import time

with open("seed.json") as f:
    data = json.load(f)

N = 5
theta = data["theta"]
clocks = data["clocks"]
queues = data["queues"]

print("\n--- CLOCK SYNCHRONIZATION START ---\n")
time.sleep(1)

T0 = clocks[0]

S = [i for i in range(N) if abs(clocks[i] - T0) <= theta]
```

```
print(f"Trusted Nodes: {S}")
time.sleep(1)

T_target = sum(clocks[i] for i in S) // len(S)

print(f"Target Time: {T_target}\n")
time.sleep(1)

delta = [0] * N
T_sync = [0] * N

for i in range(N):
    if i in S:
        delta[i] = T_target - clocks[i]
    else:
        delta[i] = 0
    T_sync[i] = clocks[i] + delta[i]
    print(f"Node {i}: Initial={clocks[i]}, Delta={delta[i]},
Synced={T_sync[i]}")
    time.sleep(0.5)

print("\n--- CLOCK SYNCHRONIZATION END ---\n")
time.sleep(1)

print("--- MESSAGE PROCESSING START ---\n")

V = [[0]*N for _ in range(N)]

buffers = [[] for _ in range(N)]

def can_process(i, sender, v_msg):
    if v_msg[sender] != V[i][sender] + 1:
        return False
    for k in range(N):
        if k != sender and v_msg[k] > V[i][k]:
            return False
    return True

def process_message(i, msg):
    sender = msg["sender"]
```

```
    v_msg = msg["v_msg"]
    print(f"Node {i} RECEIVED msg from {sender} -> {v_msg}")
    time.sleep(0.4)
    if can_process(i, sender, v_msg):
        print(f"Node {i} PROCESSED message from {sender}")
        for k in range(N):
            V[i][k] = max(V[i][k], v_msg[k])
        time.sleep(0.4)
        return True
    else:
        print(f"Node {i} BUFFERED message from {sender}")
        buffers[i].append(msg)
        time.sleep(0.4)
        return False

def try_buffer(i):
    changed = True
    while changed:
        changed = False
        for msg in buffers[i][:]:
            if can_process(i, msg["sender"], msg["v_msg"]):
                print(f"Node {i} RELEASED buffered msg from
{msg['sender']}")
                for k in range(N):
                    V[i][k] = max(V[i][k], msg["v_msg"][k])
                buffers[i].remove(msg)
                time.sleep(0.4)
                changed = True

for i in range(N):
    print(f"\nProcessing Node {i} Queue\n")
    time.sleep(0.8)

    for msg in queues[i]:
        process_message(i, msg)
        try_buffer(i)

    try_buffer(i)

print("\n--- MESSAGE PROCESSING END ---")
```

```python
time.sleep(1)

print("\n--- FINAL PAYLOADS ---\n")

P = [0]*N

for i in range(N):
    total_v = sum(V[i])
    P[i] = (T_sync[i] * total_v) % 9973
    print(f"Node {i}: V = {V[i]}, Sum = {total_v}, Payload = {P[i]}")
    time.sleep(0.5)

print("\n--- DONE ---\n")
```

Output:

```
PS C:\Users\yashv\Documents\DPC_lab\Lab8> python assignment.py

--- CLOCK SYNCHRONIZATION START ---

Trusted Nodes: [0, 1, 2, 4]
Target Time: 995

Node 0: Initial=1000, Delta=-5, Synced=995
Node 1: Initial=1020, Delta=-25, Synced=995
Node 2: Initial=950, Delta=45, Synced=995
Node 3: Initial=8000, Delta=0, Synced=8000
Node 4: Initial=1010, Delta=-15, Synced=995

--- CLOCK SYNCHRONIZATION END ---

--- MESSAGE PROCESSING START ---


Processing Node 0 Queue

Node 0 RECEIVED msg from 2 -> [1, 1, 1, 0, 0]
Node 0 BUFFERED message from 2
Node 0 RECEIVED msg from 1 -> [0, 1, 0, 0, 0]
Node 0 PROCESSED message from 1
Node 0 RECEIVED msg from 4 -> [0, 0, 0, 1, 1]
Node 0 BUFFERED message from 4
Node 0 RECEIVED msg from 3 -> [0, 0, 0, 1, 0]
Node 0 PROCESSED message from 3
Node 0 RELEASED buffered msg from 4

Processing Node 1 Queue

Node 1 RECEIVED msg from 0 -> [2, 1, 1, 0, 0]
Node 1 BUFFERED message from 0
Node 1 RECEIVED msg from 2 -> [1, 1, 1, 0, 0]
Node 1 BUFFERED message from 2
Node 1 RECEIVED msg from 0 -> [1, 0, 0, 0, 0]
Node 1 PROCESSED message from 0
Node 1 RECEIVED msg from 4 -> [0, 0, 0, 1, 1]
Node 1 BUFFERED message from 4
Node 1 RECEIVED msg from 3 -> [0, 0, 0, 1, 0]
```

```
Node 1 PROCESSED message from 3
Node 1 RELEASED buffered msg from 4

Processing Node 2 Queue

Node 2 RECEIVED msg from 4 -> [0, 0, 0, 1, 1]
Node 2 BUFFERED message from 4
Node 2 RECEIVED msg from 3 -> [0, 0, 0, 1, 0]
Node 2 PROCESSED message from 3
Node 2 RELEASED buffered msg from 4
Node 2 RECEIVED msg from 1 -> [0, 1, 0, 0, 0]
Node 2 PROCESSED message from 1
Node 2 RECEIVED msg from 0 -> [2, 1, 1, 0, 0]
Node 2 BUFFERED message from 0
Node 2 RECEIVED msg from 0 -> [1, 0, 0, 0, 0]
Node 2 PROCESSED message from 0

Processing Node 3 Queue

Node 3 RECEIVED msg from 0 -> [2, 1, 1, 0, 0]
Node 3 BUFFERED message from 0
Node 3 RECEIVED msg from 2 -> [1, 1, 1, 0, 0]
Node 3 BUFFERED message from 2
Node 3 RECEIVED msg from 0 -> [1, 0, 0, 0, 0]
Node 3 PROCESSED message from 0
Node 3 RECEIVED msg from 1 -> [0, 1, 0, 0, 0]
Node 3 PROCESSED message from 1
Node 3 RELEASED buffered msg from 2
Node 3 RELEASED buffered msg from 0

Processing Node 4 Queue

Node 4 RECEIVED msg from 2 -> [1, 1, 1, 0, 0]
Node 4 BUFFERED message from 2
Node 4 RECEIVED msg from 0 -> [1, 0, 0, 0, 0]
Node 4 PROCESSED message from 0
```

```
Node 4 RECEIVED msg from 3 -> [0, 0, 0, 1, 0]
Node 4 PROCESSED message from 3

--- MESSAGE PROCESSING END ---

--- FINAL PAYLOADS ---

Node 0: V = [0, 1, 0, 1, 1], Sum = 3, Payload = 2985
Node 1: V = [1, 0, 0, 1, 1], Sum = 3, Payload = 2985
Node 2: V = [1, 1, 0, 1, 1], Sum = 4, Payload = 3980
Node 3: V = [2, 1, 1, 0, 0], Sum = 4, Payload = 2081
Node 4: V = [2, 1, 1, 1, 0], Sum = 5, Payload = 4975

--- DONE ---
```